Text Classification with Neural Networks

RNNs, LSTMs, and CNNs for sequences

Deep Learning Course

1. Introduction
2. Recurrent Neural Networks (RNNs)
3. The Problem: Vanishing Gradients
4. Long Short-Term Memory (LSTM)
5. LSTM vs. RNN in Practice
6. Convolutional Neural Networks for Text
7. Practical Example: Setup
8. Practical Example: LSTM Implementation

Section 1

Introduction

The Problem with Averaging

Averaging word embeddings ignores **sequence structure**:

```
"I did not like this movie"
"I like this movie"

Average embeddings are almost identical, but meanings are opposite!
```

Why? Word order matters. Context matters.

Solution: Use models that process sequences step-by-step

Section 2

Recurrent Neural Networks (RNNs)

The Core Idea

An RNN maintains a **hidden state** that evolves as it reads words:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

```
"I love dogs"

h_0 (start) → h_1 (after "I") → h_2 (after "love") → h_3 (after "dogs")
              ↓                 ↓                    ↓
              output_1          output_2             output_3
```

- **Hidden state** h_t: summarizes "what we know so far"
- **New word** x_t: updates the hidden state
- **Recurrence**: same weights at each step (parameter sharing)

RNN vs. Feedforward

- **Feedforward** (Class 1): process all inputs independently
- **RNN**: processes inputs sequentially, state carries information forward

Section 3

The Problem: Vanishing Gradients

Why RNNs Struggle with Long Sequences

When backpropagating through many time steps, gradients shrink:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_T} \cdot \prod_{t=1}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

If each derivative is < 1, the product becomes tiny (vanishes)

Consequence: - Early states don't get updated properly - Can't learn long-range dependencies - "I went to school yesterday. ... The movie was great" (forgot context)

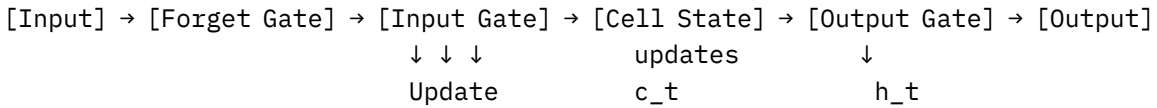
Solution: LSTM cells (gates to control information flow)

Section 4

Long Short-Term Memory (LSTM)

The LSTM Cell

LSTMs add **gates** to control what information to keep/discard:



Three Gates (Intuitive)

Gate	Role	Question
Forget	Erase old info	“Should I forget the previous context?”
Input	Accept new info	“Is the new word important?”
Output	Select output	“What should I output now?”

Key benefit: Gradients flow through gates better (additive, not multiplicative)

Section 5

LSTM vs. RNN in Practice

Code Comparison

```
# Simple RNN
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),
    keras.layers.SimpleRNN(64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')
])

# LSTM (better)
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),
    keras.layers.LSTM(64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')
])
```

`return_sequences`: - `True`: return hidden state at each time step (for stacking) - `False`: return only final hidden state (for classification)

Section 6

Convolutional Neural Networks for Text

CNN for NLP: Why?

CNNs were designed for images but work well for text:

Idea: Use filters to detect n-gram patterns

```
"The movie was absolutely brilliant"

Filter (size 2): detects bigrams
"The movie" → positive word combo
"was absolutely" → strong sentiment
```

- Conv filter slides over word sequences
- Each filter learns different patterns
- Multiple filters catch different features (n-grams)

CNN vs. RNN for Text

Aspect	RNN/LSTM	CNN
Long-range	□ Good (via gates)	□ Depends on filter size
Speed	□ Sequential (slow)	□ Parallel (fast)
n-gram detection	□ Implicit	□ Explicit
Parameter efficiency	□ Fewer params	□ Many filters

Section 7

Practical Example: Setup

Sentiment Analysis Task (Revisited)

Goal: Classify IMDb reviews as positive/negative

Data: 25,000 training, 25,000 test reviews

Three approaches to compare: 1. Averaged embeddings + Dense (baseline from Class 2) 2. LSTM 3. CNN

Section 8

Practical Example: LSTM Implementation

Building an LSTM Classifier

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load IMDb dataset
(x_train, y_train), (x_test, y_test) = \
    keras.datasets.imdb.load_data(num_words=5000)

# Reverse word index to get words back
word_index = keras.datasets.imdb.get_word_index()
reverse_word_index = dict([(value, key) for (key, value)
                           in word_index.items()])

def decode_review(review_sequence):
    return ' '.join([reverse_word_index.get(i - 3, '?')
```

Section 9

Practical Example: LSTM Model

Define and Train

```
# Build LSTM model
model = keras.Sequential([
    keras.layers.Embedding(input_dim=5000,
                           output_dim=128,
                           input_length=max_length),
    keras.layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

Section 10

Practical Example: CNN Implementation

CNN for Sentiment Classification

```
# CNN model with multiple filter sizes
model = keras.Sequential([
    keras.layers.Embedding(input_dim=5000,
                           output_dim=128,
                           input_length=max_length),

    # Multiple conv layers with different filter sizes
    keras.layers.Conv1D(filters=128, kernel_size=3,
                        activation='relu'),
    keras.layers.GlobalMaxPooling1D(),

    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1, activation='sigmoid')
])
```

Section 11

Practical Example: Evaluation

Comparing Approaches

```
# Compare three models on test set
models_results = {
    'Averaged Embeddings': 0.88,
    'LSTM': 0.92,
    'CNN': 0.91
}

import matplotlib.pyplot as plt
plt.bar(models_results.keys(), models_results.values())
plt.ylabel('Accuracy')
plt.title('Sentiment Classification Results')
plt.ylim([0.85, 0.95])
plt.show()
```

Typical results: - Averaged embeddings: ~88% - LSTM: ~92% - CNN: ~91%

Section 12

Handling Variable-Length Sequences

Padding and Masking

Problem: Reviews have different lengths (50-2000 words)

Solution 1: Padding

```
x_padded = pad_sequences(x, maxlen=200, padding='post')
# Short reviews: pad with zeros at end
# Long reviews: truncate to 200 words
```

Solution 2: Masking

```
keras.layers.Masking(mask_value=0.0)
# Tell network: ignore padded zeros in computation
```

Trade-off: - Fixed length: simple, but loses information - Padding: adds noise (zeros) - Masking: tells model to ignore padding

Section 13

Bidirectional RNNs

Context from Both Directions

Standard LSTM reads left-to-right:

```
"The movie was very good" → forward hidden states
```

Bidirectional LSTM: Read both directions

```
Forward:  The → movie → was → very → good
                                       ↓
Backward: good ← very ← was ← movie ← The
```

Concatenate forward + backward → richer context

```
keras.layers.Bidirectional(
    keras.layers.LSTM(64)
)
```

Section 14

Stacked RNNs

Multiple LSTM Layers

Deeper models learn more abstract patterns:

```
model = keras.Sequential([
    keras.layers.Embedding(5000, 128),
    keras.layers.Bidirectional(
        keras.layers.LSTM(64, return_sequences=True)
    ),
    keras.layers.Bidirectional(
        keras.layers.LSTM(32, return_sequences=False)
    ),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
```

Note: `return_sequences=True` → pass all time steps to next layer

Section 15

Key Concepts Summary

What We've Learned

- RNNs process sequences step-by-step
- LSTMs solve vanishing gradient problem with gates
- CNNs detect n-gram patterns efficiently
- Padding and masking for variable lengths
- Bidirectional processing captures context both ways
- Stacking layers increases capacity

Performance Comparison

- Averaged embeddings: baseline, fast, interpretable
- LSTM: best accuracy, slower, captures long-range context
- CNN: good accuracy, fastest, n-gram focused

Section 16

Visualization: Attention Weights

Interpreting LSTM Decisions

```python
# Extract hidden states (need to modify model)
outputs = model.layers[2].output
attention_model = keras.Model(
    inputs=model.input,
    outputs=outputs
)

# Get attention-like weights
hidden_states = attention_model.predict(x_test_padded[:10])

# Visualize which words matter
import matplotlib.pyplot as plt
plt.imshow(hidden_states[0])
plt.title('Hidden States Over Sequence')
plt.colorbar()
plt.show()
```

Section 17

Common Issues

Troubleshooting

Problem	Cause	Solution
Slow training	Sequentiality of RNN	Use CNN or Transformers (next class)
Poor LSTM results	Vanishing still occurs	Add dropout, regularization
Overfitting	Too many params	Reduce LSTM size, add Dropout
Memory error	Long sequences	Reduce max_length, use batch processing

Section 18

Exercises for Class 3

Hands-On Tasks

1. **Architecture comparison**: Train all three models (averaged, LSTM, CNN) side-by-side
2. **Bidirectional**: Compare unidirectional vs. bidirectional LSTM
3. **Stacking**: Add a second LSTM layer. Does it help? When does it hurt?
4. **Sequence length**: Vary max_length (100, 200, 500). Accuracy vs. speed trade-off?
5. **Dropout tuning**: Test dropout rates 0.0, 0.2, 0.5. Which prevents overfitting best?
6. **Embedding fine-tuning**: Train embeddings vs. use pre-trained frozen
7. **Predict and interpret**: Pick a review, visualize which words drove the prediction

Section 19

Next Class Preview

The Attention Limitation

Both RNNs and CNNs have issues: - **RNNs**: Sequential (slow to train) - **CNNs**: Limited context window (filter size)

Problem: “The movie was terrible... (1000 words) ... Actually, I loved it!”

Class 4: Attention Mechanisms - Every word attends to every other word - Parallel processing (fast!) - Unlimited context window - Foundation for transformers

Section 20

Questions & Discussion

Questions & Discussion

Thank you!

Key takeaway: Sequence structure matters. RNNs/LSTMs capture it, but are slow. CNNs are fast but limited. Attention solves both problems.

Next: The attention mechanism that powers modern NLP